

DATABASE SYSTEMS LAB

Star Schema vs Snowflake Schema

Data Warehousing in PostgreSQL

DBMS

PostgreSQL (pgAdmin 4)

SCENARIO

Retail Sales Data Warehouse

DATE

June 2026

Table of Contents

1. Objective
2. Introduction
3. **Comparison: Star vs Snowflake Schema**
 - 3.1 Feature-by-Feature Comparison
 - 3.2 Structural Diagram
4. **When to Prefer Each Schema**
 - 4.1 Prefer Star Schema When
 - 4.2 Prefer Snowflake Schema When
5. **Scenario Description**
6. **Part A — Snowflake Schema**
 - 6.1 Tables Created (10 tables)
 - 6.2 Sample Data — Product Hierarchy
 - 6.3 Sample Data — Geography Hierarchy
7. **Part B — Star Schema**
 - 7.1 Tables Created (5 tables)
 - 7.2 Denormalization Example
8. **Query Comparison & Expected Results**
 - 8.1 Sales by Category
 - 8.2 Sales by Country
 - 8.3 Side-by-Side Summary
9. **pgAdmin 4 Implementation Steps**
10. **Conclusion**
11. **Appendix — File List**

01 Objective

The objective of this lab is to explore and contrast two fundamental dimensional data modelling approaches used in modern data warehousing:

- Compare **star schema** and **snowflake schema** dimensional data models in depth.
- Identify business situations where each schema is the preferred choice.
- Design and implement both schemas for the *same* business scenario using **PostgreSQL**.
- Insert sample data and run analytical queries to demonstrate the practical differences.

02 Introduction

A **data warehouse** stores historical data for business intelligence and decision-making. Unlike operational (OLTP) databases optimised for transactions, warehouses use **dimensional modelling** — organising data into two principal building blocks:

- **Fact tables** — store measurable business events (sales amount, quantity sold, revenue).
- **Dimension tables** — store descriptive context (product, customer, store, date).

Two widely used dimensional models are compared throughout this report:

Model	Description
Star Schema	Dimensions are denormalized (flat). The fact table sits at the centre with dimension tables radiating outward
Snowflake Schema	Dimensions are normalized into multiple related tables. Hierarchies branch out from the fact table like a snow

Both models answer the same business questions; they differ in structure, storage efficiency, and query complexity. The choice between them depends on the specific workload and governance requirements.

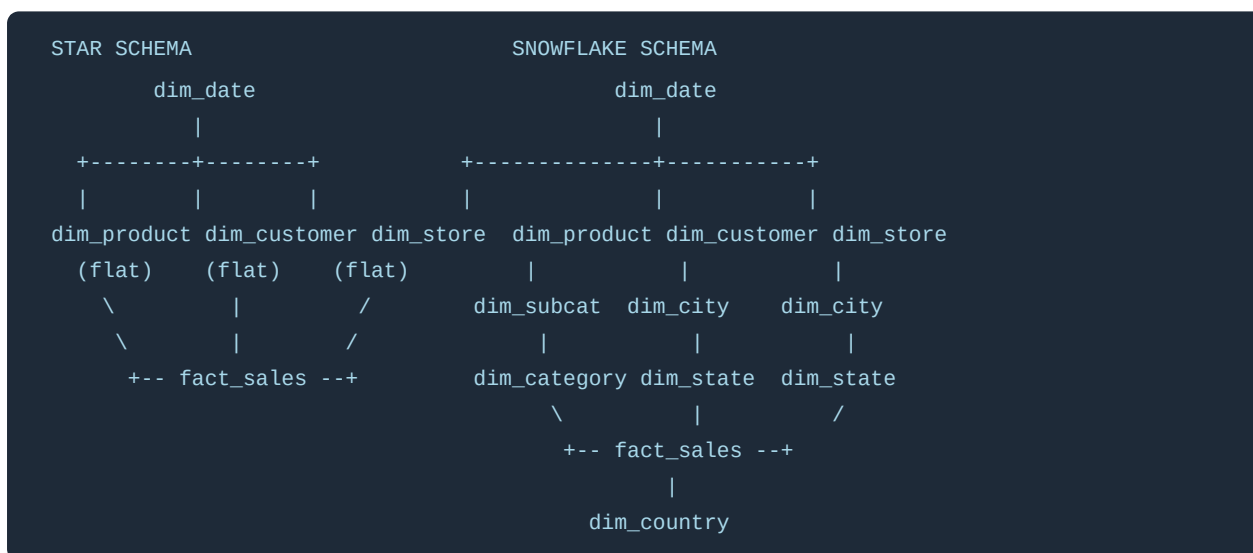
03 Comparison: Star vs Snowflake Schema

3.1 Feature-by-Feature Comparison

#	Aspect	Star Schema	Snowflake Schema
1	Structure	Fact + flat dimension tables	Fact + normalized hierarchies
2	Normalization	Denormalized (1NF per dim)	Higher normalization (3NF)
3	Table count	Fewer tables	More tables
4	Joins in queries	Fewer joins	More joins (sub-dimensions)
5	Query performance	Faster for analytics	Slower (additional joins)
6	Storage	More redundancy	Less redundancy
7	Data consistency	Risk of label drift	Single source of truth
8	ETL complexity	Simpler load; harder updates	Complex load; easy updates
9	Understandability	Easy for analysts & BI tools	Requires relational knowledge
10	Best suited for	Dashboards, OLAP, ad-hoc	Enterprise, catalogs, MDM

3.2 Structural Diagram

The diagrams below illustrate how the two schemas arrange tables around a central fact table. The star schema has direct one-hop joins while the snowflake schema introduces sub-dimension layers.



04 When to Prefer Each Schema

4.1 Prefer Star Schema When...

Situation	Reason
BI dashboards (Power BI, Tableau)	Fewer joins = faster query response
Read-heavy workloads	Optimised for SELECT aggregations
Simple / shallow hierarchies	Denormalization wastes little space
Non-technical end users	Flat tables are easier to understand
Prototyping a data warehouse	Faster to build and query

Example: A marketing team needs a daily sales dashboard grouped by product category and country — star schema delivers sub-second response times.

4.2 Prefer Snowflake Schema When...

Situation	Reason
Deep product / geography hierarchies	Normalisation avoids massive duplication
Frequent dimension updates	Change category name in one row, not thousands
Strict data governance / MDM	Enforces referential integrity at every level
Storage cost is a concern	Eliminates redundant text in dimension tables
Integration with normalised OLTP sources	Easier mapping from source relational schema

Example: A global retailer with 50,000 products across 200 subcategories that are reorganised quarterly — snowflake ensures one-place updates with zero inconsistency.

05 Scenario Description

Business case: A retail company tracks sales transactions. Each sale records five key dimensions plus two quantitative measures:

- **What** was sold — product
- **Who** bought it — customer
- **Where** it was sold — store
- **When** it happened — date

- **How much** was sold — quantity and total amount

Dimensions & Hierarchies

Dimension	Snowflake Hierarchy	Star (Flattened Attributes)
Product	Category → Subcategory → Product	category_name, subcategory_name, product_name
Customer	Country → State → City → Customer	country_name, state_name, city_name, customer_name
Store	Country → State → City → Store	country_name, state_name, city_name, store_name
Date	Year → Quarter → Month → Day	year_num, quarter, month_num, day_num

Fact Table — fact_sales

One row per sale transaction with foreign keys to all four dimension tables, plus measures **quantity** (integer) and **total_amount** (numeric). This table is *identical* in both schemas; only the dimension table structure differs.

06

Part A — Snowflake Schema

The snowflake schema normalises every dimension hierarchy into separate lookup tables. This produces **10 tables** in the retail_snowflake_dw database.

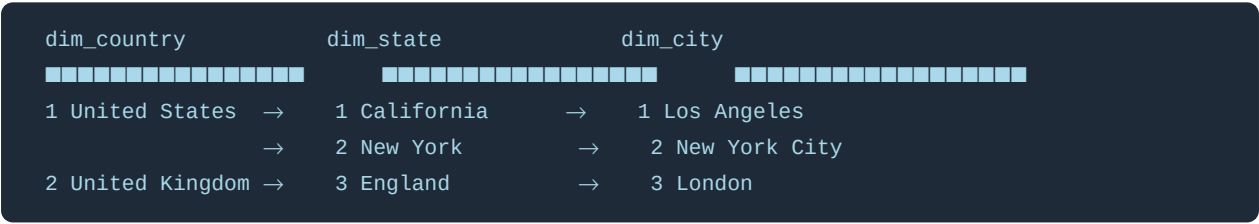
6.1 Tables Created (10 tables)

Table	Type	Rows	Description
dim_category	Dimension L1	3	Electronics, Clothing, Groceries
dim_subcategory	Dimension L2	6	Laptops, Smartphones, Mens Wear, etc.
dim_product	Dimension L3	8	Individual products with unit prices
dim_country	Dimension L1	2	United States, United Kingdom
dim_state	Dimension L2	3	California, New York, England
dim_city	Dimension L3	3	Los Angeles, New York City, London
dim_customer	Dimension	4	Customers linked to cities
dim_store	Dimension	3	Stores linked to cities
dim_date	Dimension	5	Date attributes for Q1–Q2 2024
fact_sales	Fact	10	Core sales transactions

6.2 Sample Data — Product Hierarchy

dim_category		dim_subcategory		dim_product
1 Electronics	→	1 Laptops	→	1 Dell XPS 15
	→	2 Smartphones	→	3 iPhone 15
2 Clothing	→	3 Mens Wear	→	5 Mens Formal Shirt
	→	4 Womens Wear	→	6 Womens Summer Dress
3 Groceries	→	5 Beverages	→	7 Orange Juice 1L
	→	6 Snacks	→	8 Potato Chips 200g

6.3 Sample Data — Geography Hierarchy



07 Part B — Star Schema

The star schema collapses every dimension hierarchy into a single flat table. This produces only **5 tables** in the retail_star_dw database, with the same 10 fact rows.

7.1 Tables Created (5 tables)

Table	Type	Rows	Description
dim_product	Dimension (denormalized)	8	Includes category + subcategory in same row
dim_customer	Dimension (denormalized)	4	Includes city + state + country in same row
dim_store	Dimension (denormalized)	3	Includes city + state + country in same row
dim_date	Dimension	5	Same dates as snowflake schema
fact_sales	Fact	10	Identical transactions as snowflake schema

7.2 Denormalization Example — dim_product

Instead of three linked tables (dim_category → dim_subcategory → dim_product), a single flat table holds all attributes. Hierarchy values are **repeated** in every product row.

product_id	product_name	subcategory_name	category_name	unit_price
1	Dell XPS 15	Laptops	Electronics	\$1,299.99
5	Mens Formal Shirt	Mens Wear	Clothing	\$49.99
7	Orange Juice 1L	Beverages	Groceries	\$3.99

08 Query Comparison & Expected Results

Both schemas contain the **same 10 sales transactions** totalling **\$6,856.82**. The analytical results are identical; only the SQL complexity differs.

8.1 Sales by Category

Snowflake (3 joins required):

```

SELECT c.category_name, SUM(f.total_amount) AS total_sales
FROM fact_sales f
JOIN dim_product p      ON f.product_id = p.product_id
JOIN dim_subcategory sc ON p.subcategory_id = sc.subcategory_id
JOIN dim_category c     ON sc.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_sales DESC;
```

Star (1 join required):

```

SELECT p.category_name, SUM(f.total_amount) AS total_sales
FROM fact_sales f
JOIN dim_product p ON f.product_id = p.product_id
GROUP BY p.category_name
ORDER BY total_sales DESC;
```

Expected Output (both schemas):

category_name	total_sales
Electronics	\$6,598.92
Clothing	\$229.96
Groceries	\$27.42

8.2 Sales by Country

Snowflake (4 joins required):

```

SELECT co.country_name, SUM(f.total_amount) AS total_sales
```

```

FROM fact_sales f
JOIN dim_store s    ON f.store_id = s.store_id
JOIN dim_city ci    ON s.city_id = ci.city_id
JOIN dim_state st   ON ci.state_id = st.state_id
JOIN dim_country co ON st.country_id = co.country_id
GROUP BY co.country_name;
    
```

Star (1 join required):

```

SELECT s.country_name, SUM(f.total_amount) AS total_sales
FROM fact_sales f
JOIN dim_store s ON f.store_id = s.store_id
GROUP BY s.country_name;
    
```

Expected Output (both schemas):

country_name	total_sales
United States	\$5,676.84
United Kingdom	\$1,179.98

8.3 Side-by-Side Summary

Metric	Snowflake Schema	Star Schema
Total tables	10	5
Joins for category report	3	1
Joins for country report	4	1
Total sales amount	\$6,856.82	\$6,856.82
Fact table rows	10	10

09 pgAdmin 4 Implementation Steps

Follow these steps to implement the lab in **pgAdmin 4** connected to a local PostgreSQL instance.

Step 1 — Create Databases

Open pgAdmin 4, connect to your PostgreSQL server, right-click **Databases** → **Query Tool** (opens on the postgres database), then open and execute `setup_databases.sql`. Refresh the database list — you should see `retail_snowflake_dw` and `retail_star_dw`.

Step 2 — Implement Snowflake Schema

Right-click `retail_snowflake_dw` → **Query Tool**. Open `snowflake_schema.sql` and click **Execute (F5)**. Verify by expanding **Schemas** → **public** → **Tables** — 10 tables should appear. Right-click `fact_sales` → View/Edit Data → All Rows to confirm 10 records.

Step 3 — Implement Star Schema

Right-click `retail_star_dw` → **Query Tool**. Open `star_schema.sql` and execute. Verify: 5 tables should appear with the same 10 fact rows.

Step 4 — View ERD in pgAdmin

Right-click the database → **ERD For Database**. pgAdmin generates a visual diagram of all tables and foreign-key relationships. Take a screenshot for your lab submission.

Step 5 — Run Verification Queries

The verification queries at the bottom of each SQL file run automatically on execute. Check the **Data Output** tab for results matching Section 8.

Troubleshooting

Problem	Solution
database already exists	Run <code>setup_databases.sql</code> first (it drops and recreates)
relation already exists	Each script drops tables first; re-run the full file
permission denied to create database	Use a superuser account (default: postgres)
Cannot drop database (active connections)	<code>setup_databases.sql</code> terminates connections automatically

10 Conclusion

This lab demonstrated two fundamental dimensional modelling approaches using the same retail sales scenario implemented in PostgreSQL:

Schema	Tables	Join Depth	Primary Advantage
Snowflake	10 normalised	Up to 4 hops	Data consistency, storage efficiency, hierarchy updates
Star	5 denormalized	1 hop	Query simplicity, faster analytics, BI-tool friendly

Key Takeaway: There is no universally "better" schema. Choose STAR when query performance and simplicity matter most. Choose SNOWFLAKE when data integrity, hierarchy maintenance, and storage efficiency are priorities. In practice, many organisations normalise during ETL and materialise star-schema views for end-user reporting — combining the benefits of both approaches.

11 Appendix — File List

The following SQL and documentation files accompany this lab report:

File	Purpose
FINAL_REPORT.md	This report — submit as lab documentation
setup_databases.sql	Creates both PostgreSQL databases (run on postgres DB first)
snowflake_schema.sql	Snowflake schema: tables, sample data, verification queries
star_schema.sql	Star schema: tables, sample data, verification queries
README.md	Quick reference guide for the lab

End of Report — Database Systems Lab, June 2026